Feature Detection & Matching

Week 10 — Session 1: Corners & descriptors — finding distinctive points

Practical Image Processing

Lecture 10.1 ▪ 60–70 min

Recap & today

- Wk5–9: edges, regions, shapes — describing an image's content.
- This session: **features** — distinctive points (corners) and how to turn each into a **descriptor** vector.
- Next session: **matching** those descriptors across images and rejecting bad matches.

DocScan milestone

Finding repeatable points on a photographed page is the first step to aligning it to a template — the setup for Week 11's perspective correction.

Roadmap — Session 1

What makes a good feature

The Harris corner detector

From keypoint to descriptor

In-class exercise

Wrap-up

What makes a good feature

What is a good feature point?

- **Distinctive**: it looks different from its surroundings (so it can be found again).
- **Repeatable**: the same physical point is detected across images (different view/light).
- **Local**: described by a small patch, robust to partial occlusion.

Key idea

Flat regions and straight edges are ambiguous; **corners** are the classic good feature — they pin down a location in 2D.

Flat / edge / corner — the intuition

- **Flat**: slide the window anywhere → patch unchanged → no localization.
- **Edge**: slide *along* the edge → unchanged → 1D ambiguity.
- **Corner**: slide *any* direction → patch changes → pinned in 2D.



Math

The **Harris** detector turns "does the patch change if I shift it?" into linear algebra on the image gradients.

The sliding-window energy

- Shift a window by (u, v) and measure how much the patch changes:

$$E(u,v) = \sum_{(x,y)\in W} w(x,y)\,\big[I(x+u, y+v) - I(x,y)\big]^2.$$

- First-order Taylor: $I(x+u, y+v) \approx I(x,y) + I_x u + I_y v$, so

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}, \qquad M = \sum_W w \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}.$$

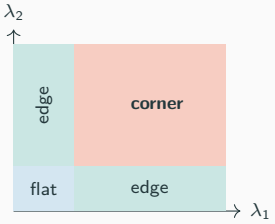
Key idea

The change energy is a quadratic form in the shift. The matrix M (the **structure matrix**) encodes all directions at once.

The Harris corner detector

The structure matrix, intuitively

- Look at how image gradients (I_x, I_y) vary inside a small window.
- The two eigenvalues λ_1, λ_2 of M say how much the patch changes in the two principal directions.



Math

Both small $\rightarrow$ flat. One large $\rightarrow$ edge. Both large $\rightarrow$ corner.

Harris score — avoiding the eigenvalues

- Computing λ_1, λ_2 per pixel is costly. Harris uses determinant and trace instead:

$$R = \det(M) - k\left[\mathrm{tr}(M)\right]^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2, \quad k \approx 0.04\text{–}0.06.$$

- R large positive $\rightarrow$ corner. $R < 0 \rightarrow$ edge. $|R|$ small $\rightarrow$ flat.

Math

$\det(M) = I_x^2 I_y^2 - (I_x I_y)^2$ and $\mathrm{tr}(M) = I_x^2 + I_y^2$ (summed over the window) — both are cheap to accumulate.

Worked example — the Harris score

Suppose over a window we summed:

$$\sum I_x^2 = 5, \quad \sum I_y^2 = 4, \quad \sum I_x I_y = 1, \quad k = 0.05.$$

$$\begin{aligned} \det(M) &= (5)(4) - (1)^2 = 19, \\ \operatorname{tr}(M) &= 5 + 4 = 9, \\ R &= 19 - 0.05\,(9)^2 \\ &= 19 - 4.05 = \mathbf{14.95}. \end{aligned}$$

Reading it

$R \gg 0$: both gradient directions are strong $\Rightarrow$ a **corner**.

Contrast: an edge with $\sum I_x^2 = 9$, $\sum I_y^2 = 0$, $\sum I_x I_y = 0$ gives $\det = 0$, $R = -0.05(81) = -4.05 < 0$.

Harris corners — on a real car



grayscale



Harris corners (red)

Corners land on distinctive junctions (lights, edges of parts) — repeatable points to match on

The Harris algorithm, step by step

1. Compute gradients I_x, I_y (e.g. Sobel).
2. Form the products $I_x^2,\ I_y^2,\ I_x I_y$ at every pixel.
3. Smooth each product with a Gaussian window w (this is the $\sum_W$).
4. Compute R per pixel from det and tr.
5. Threshold R, then apply **non-maximum suppression** to keep local peaks.

✔ Good practice

Gaussian weighting (not a box) makes the response isotropic and less noisy — the standard choice.

Non-maximum suppression

- A strong corner fires over a small blob of pixels — we want *one* keypoint, not many.
- Keep a pixel only if its R is the local maximum in a small neighbourhood.
- Then take the top-N by score for a manageable, well-spread set.

2	4	3
5	**9**	6
3,7,4		

Center 9 is the max of its 3×3 neighbourhood $\Rightarrow$ kept; the others are suppressed.

Corner detectors

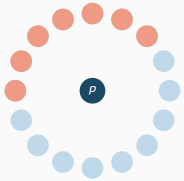
Detector	Idea	Note
Harris	eigenvalues of structure matrix	classic, accurate
Shi-Tomasi	min eigenvalue (goodFeaturesToTrack)	great for tracking
FAST	pixel-ring intensity test	very fast, used in ORB

Key idea

Detecting *where* the feature is (the keypoint) is step one. Describing *what it looks like* (the descriptor) is step two.

FAST — a corner test you can do by hand

- Look at a ring of 16 pixels around a candidate p.
- p is a corner if ≥ 12 contiguous ring pixels are all *brighter* than $I_p + t$ or all *darker* than $I_p - t$.
- A quick test of pixels 1,5,9,13 rejects most non-corners instantly.



Coral arc = a contiguous run brighter than $I_p + t$.

Shi-Tomasi — a small but useful tweak

- Harris scores with $R = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$; Shi-Tomasi scores with $R = \min(\lambda_1, \lambda_2)$.
- A point is a good corner only if the *weaker* direction is still strong — exactly what $\min(\lambda_1, \lambda_2)$ measures.
- OpenCV exposes it as `goodFeaturesToTrack` — the go-to for corners to *track* in video.

Same M, different score

Both read the structure matrix; they differ only in how they collapse its eigenvalues into one number.

Reading the gradients of a corner

A bright square in the lower-left of a window; its (approximate) gradient magnitudes:

0	0	0
0	90	80
0,85,10		

- Vertical edge $\rightarrow$ large $\sum I_x^2$.
- Horizontal edge $\rightarrow$ large $\sum I_y^2$.
- Both large in one window $\Rightarrow$ big $\det(M)$ $\Rightarrow$ high R: a corner.

Strong gradients appear along *both* a vertical and a horizontal edge.

Key idea

A corner is literally where two differently-oriented edges meet inside one window.

Choosing detector parameters

✔ Good practice

- **Window / σ**: larger = smoother, more stable corners but coarser localization.
- **Threshold**: set it from the score distribution (e.g. a fraction of the max R), not a magic constant.
- **Max features + NMS radius**: cap the count and spread points out for a usable, even set.

Measure success by **repeatability**: run the detector on the image and a warped copy, and check how many of the same points reappear.

Pitfalls in corner detection

✕ Pitfall

- **No smoothing**: raw gradients are noisy → spurious "corners" on texture.
- **Wrong k / threshold**: too high misses real corners; too low floods the image.
- **Skipping NMS**: you get dense clumps of near-duplicate keypoints.

Key idea

Detector quality = repeatability. Tune so the *same* points reappear across views, not just "many" points.

From keypoint to descriptor

From keypoint to descriptor

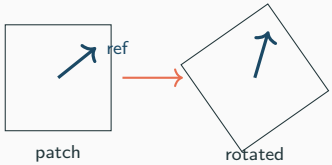
- A **descriptor** is a vector summarizing the patch around a keypoint.
- Good descriptors are **invariant**: same value under rotation, scale, and lighting change.
- Two images are compared by finding keypoints with *similar descriptors* (next session).

Math

SIFT: a 128-D vector of gradient-orientation histograms; robust to rotation/scale/illumination.

What “invariance” buys you

- **Rotation**: assign the patch a dominant orientation, then describe *relative* to it.
- **Scale**: describe at the feature’s characteristic scale (Session 2’s pyramid).
- **Illumination**: normalize the patch (subtract mean, divide by norm).



Same content, same descriptor — because we measure relative to the reference arrow.

Patch normalization — a mini worked example

A tiny patch (before / after subtract-mean, divide-by-norm):

10	20
30,40	

mean $= 25$; subtract it $\rightarrow -15, -5, 5, 15$.

-.6	-.2
.2,.6	

$\div$ norm $\approx 25.5 \rightarrow$ brightness-invariant.

✓ Good practice

A brighter copy of the same patch (add a constant, scale) normalizes to the *same* vector — that is illumination invariance, for free.

ORB keypoints — on the real car



- **ORB** = FAST keypoints + BRIEF descriptor, made rotation-aware (Session 2)

In-class exercise

In-class exercise (10 min)

Reason about detection

1. Why is a corner a better feature than a point in the middle of a flat wall?
2. A window gives $\sum I_x^2 = 6,\ \sum I_y^2 = 6,\ \sum I_x I_y = 0,\ k = 0.05$. Compute R. Corner, edge, or flat?
3. You detect 5000 corners packed into clumps. Which step did you forget, and what does it fix?

Debrief

- (1) A corner changes in *all* directions $\rightarrow$ uniquely localizable; a flat point is ambiguous.
- (2) $\det = 36 - 0 = 36$, $\mathrm{tr} = 12$, $R = 36 - 0.05(144) = 36 - 7.2 = \mathbf{28.8} > 0 \Rightarrow$ **corner**.
- (3) **Non-maximum suppression** — it collapses each response blob to a single local-peak keypoint.

Key idea

Detect where (Harris/FAST) $\rightarrow$ suppress duplicates $\rightarrow$ describe invariantly. Matching comes next session.

Wrap-up

What you should now be able to do

- Explain what makes a good feature (distinctive, repeatable, local) and why corners.
- Derive the structure matrix and compute the Harris score R by hand.
- Detect corners (Harris/Shi-Tomasi/FAST) and apply non-maximum suppression.
- Explain what a descriptor is and why invariance matters.

Next session

Session 2 — Matching & outlier rejection

- How SIFT and ORB actually build their descriptor vectors.
- Matching by Euclidean vs **Hamming** distance; brute-force vs FLANN.
- The Lowe **ratio test**, cross-check, and a preview of RANSAC.
- The week's deliverable — detect, describe, match, and filter on real images.

A corner is “a point you can find again.”

Detect distinctive points, suppress duplicates, and get them ready to describe.